

ЦЕНТРАЛЬНЫЙ
УНИВЕРСИТЕТ

Буферный кеш и журнал предзаписи

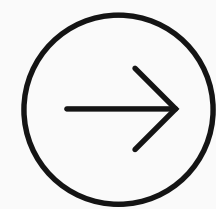
SQL И БАЗЫ ДАННЫХ

НЕДЕЛЯ 14

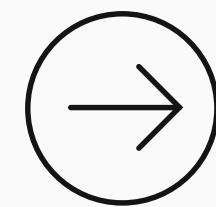
План лекции

- Иерархия памяти и конфликт I/O
- Внутреннее устройство Buffer Pool: страницы, кадры, метаданные
- ACID-I/O: политики Steal и No-Force
- Тюнинг Autovacuum
- Протокол Write-Ahead Logging (WAL) и синхронизация памяти с диском
- Восстановление после сбоев: защита от Torn Pages и протокол ARIES

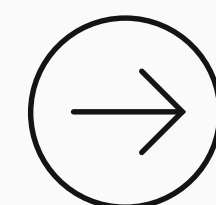
Иерархия памяти



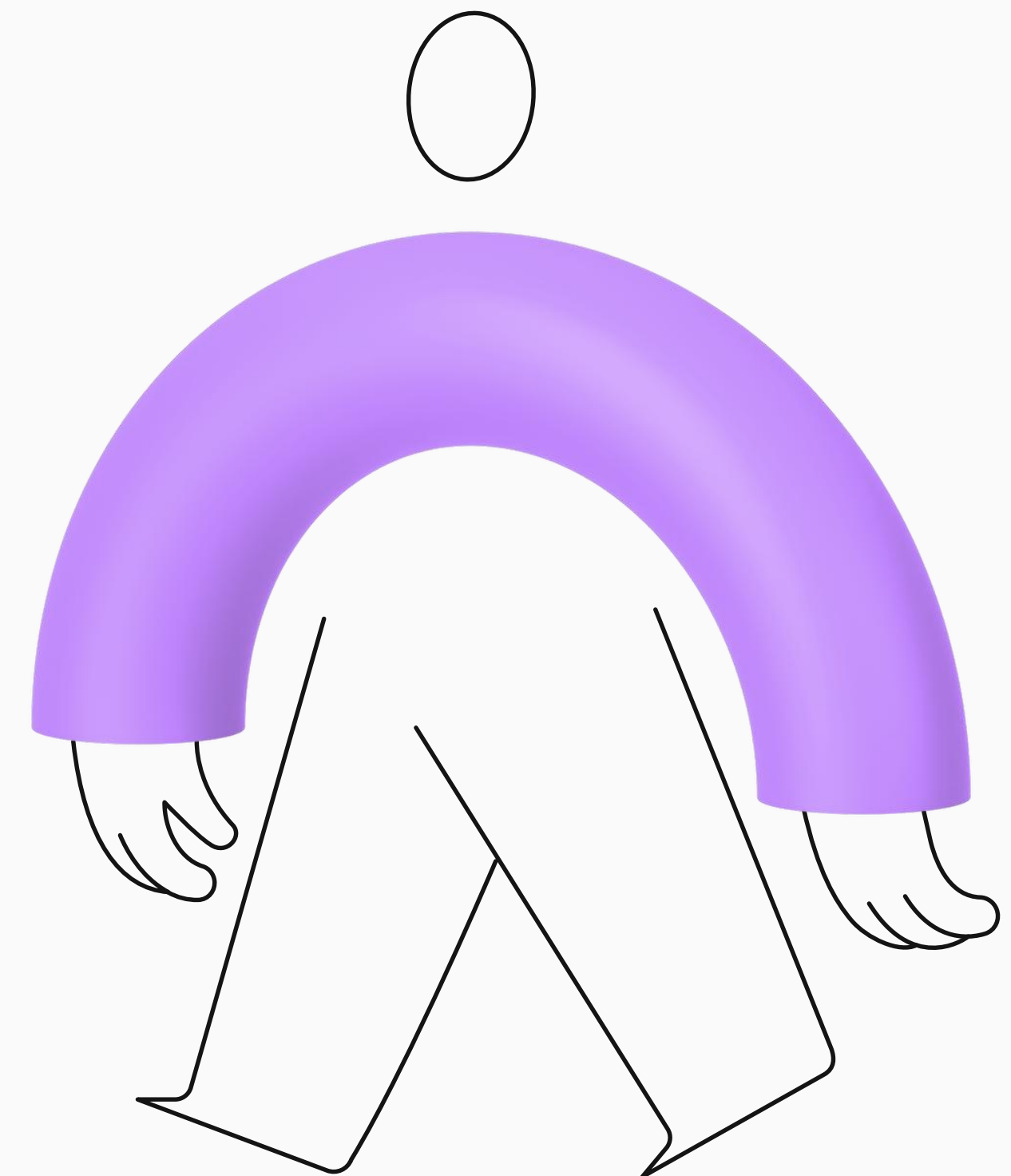
RAM (оперативная память): доступ измеряется в наносекундах (~100 нс). Данные волатильны (исчезают при отключении питания).



SSD/NVMe (диск): доступ измеряется в микросекундах (~10–100 мкс). Это в 100–1000 раз медленнее RAM. Данные персистентны.

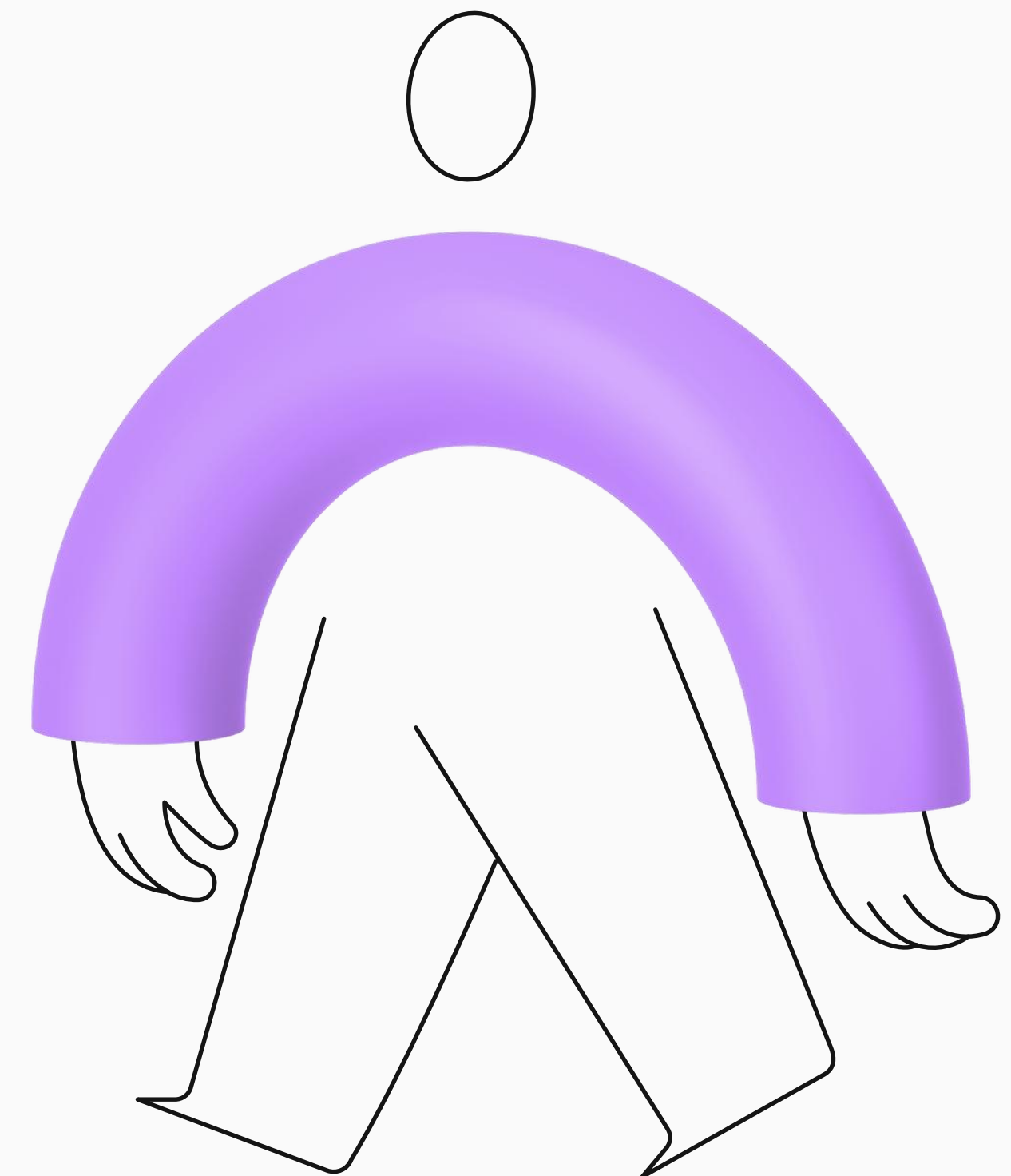


HDD (магнитный диск): доступ в миллисекундах (~10 мс). В 100 000 раз медленнее RAM.



Почему СУБД не доверяет OS Page Cache?

- 01 Проблема двойного кэширования: данные дублируются в памяти СУБД и в памяти ОС, впустую сжигая ценную RAM.
- 02 Потеря контроля над вытеснением: ОС не понимает семантики запросов. Она может вытеснить из памяти критически важный корень B-дерева, оставив там мусор от недавнего сканирования таблицы.
- 03 Нарушение ACID: ОС может сбросить страницу данных на диск до того, как запишет соответствующий лог-файл, что разрушит возможность консистентного восстановления.



Внутреннее устройство Buffer Pool

Buffer Pool — это гигантский предвыделенный массив пустых кадров в RAM, в которые СУБД временно загружает страницы с диска для чтения или модификации.

Page (страница)

Блок данных фиксированного размера (например, 8 Кб в PostgreSQL или 16 Кб в InnoDB).

Физически хранится на диске.

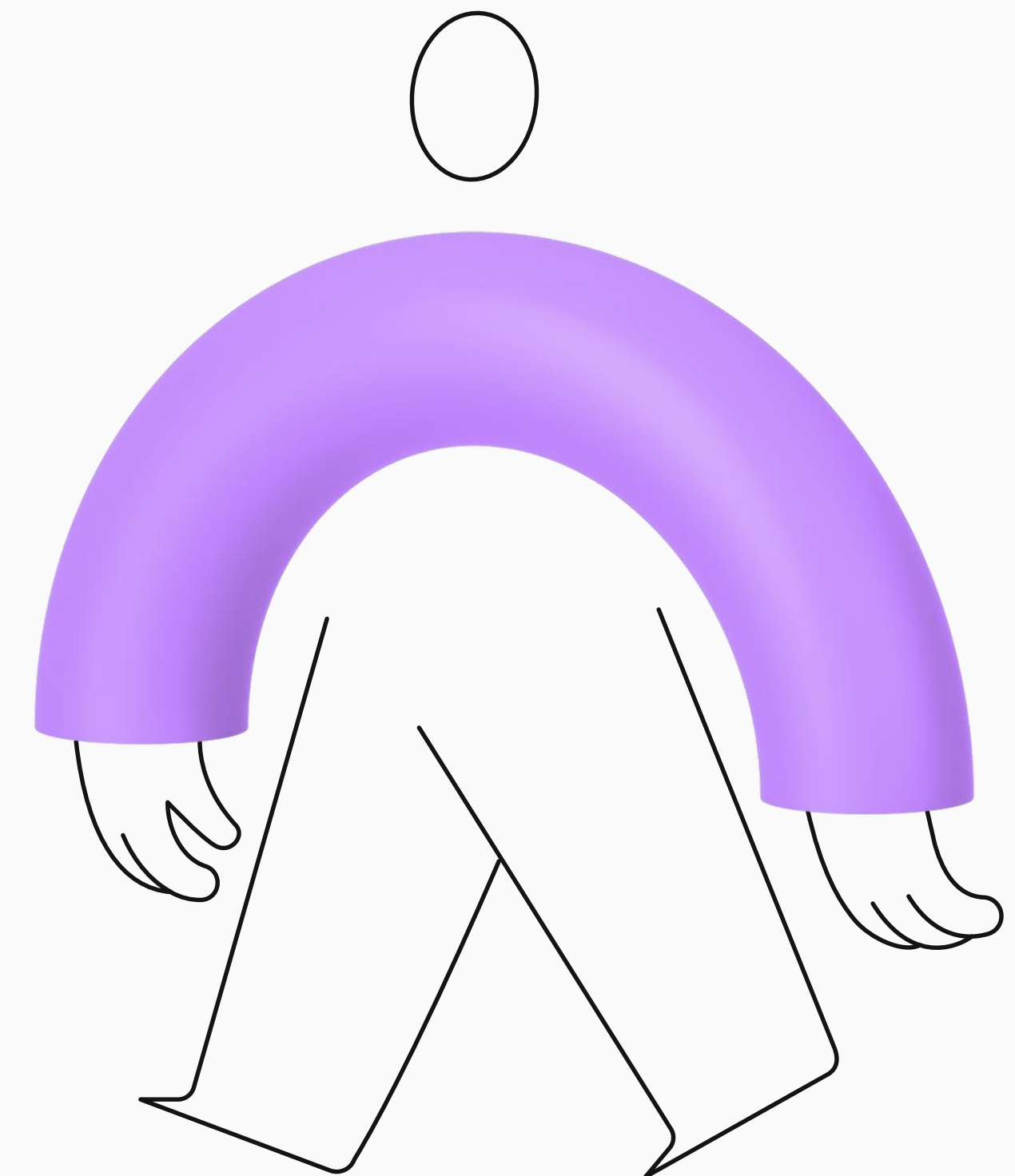
Frame (кадр)

Ячейка (слот) в оперативной памяти ровно такого же размера, предназначенная для размещения ровно одной страницы.

Метаданные и O(1) доступ

- 01 Page Table: глобальная хеш-таблица в оперативной памяти.
- 02 Маппинг: преобразует идентификатор дисковой страницы (Page_ID) в номер кадра в памяти (Frame_ID).
- 03 Cache Hit: если страница есть в таблице, возвращается указатель на кадр в памяти. Диск не трогаем.
- 04 Cache Miss: если страницы нет, СУБД находит свободный кадр, читает страницу с диска и обновляет таблицу.

* Для сверхбыстрого конкурентного доступа таблица защищена легковесными «защёлками» (Latches) на уровне потоков, а не транзакционными блокировками (Locks).



Latches vs Locks

Locks

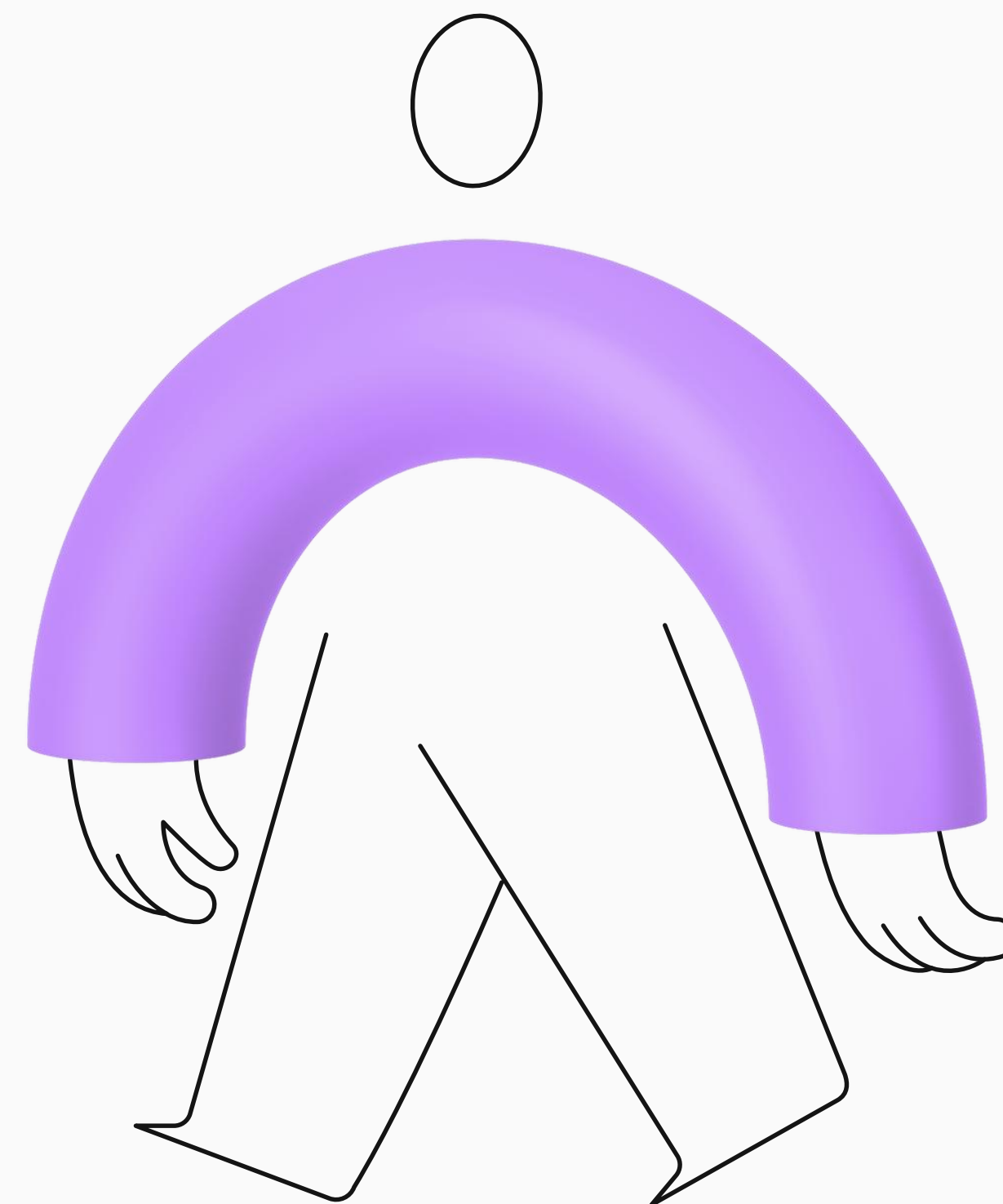
- Защищают логическое содержимое базы данных (строки, таблицы) от других пользовательских транзакций.
- Удерживаются долго (до команды COMMIT).
- Поддерживают механизмы обнаружения дедлоков.

Latches

- Защищают физические структуры памяти (Page Table, сами кадры) от других внутренних потоков (threads) самой СУБД.
- Удерживаются микросекунды (на время чтения / записи байтов в память).
- Не имеют защиты от дедлоков.

Состояния кадра и Pin Count

- 01 Free — кадр пуст, готов немедленно принять новую страницу с диска.
- 02 Pinned — в данный момент страница читается или изменяется потоком исполнения. Вытеснять её на диск категорически запрещено. Контролируется счётчиком обращений (Pin Count). Кадр можно вытеснить, только когда Pin Count = 0.
- 03 Dirty — страница была изменена в оперативной памяти, и её содержимое теперь отличается от того, что хранится на диске. Перед вытеснением такого кадра его обязательно нужно записать на накопитель.

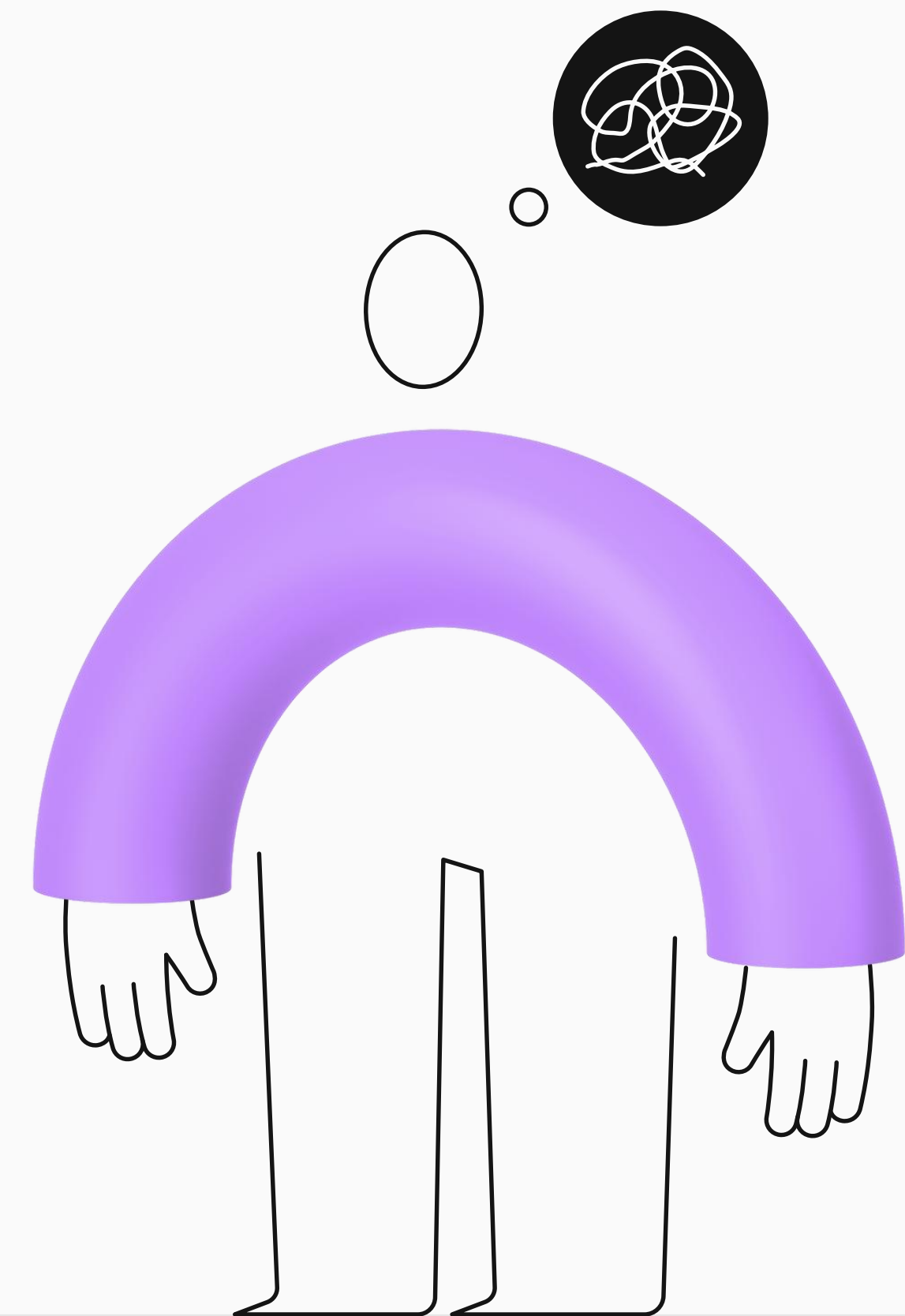


Классический LRU

Что делать системе, если уперлись в лимит RAM, а для выполнения запроса нужно загрузить новую страницу?

Политика LRU (Least Recently Used): вытесняем страницу, к которой дольше всего не было обращений.

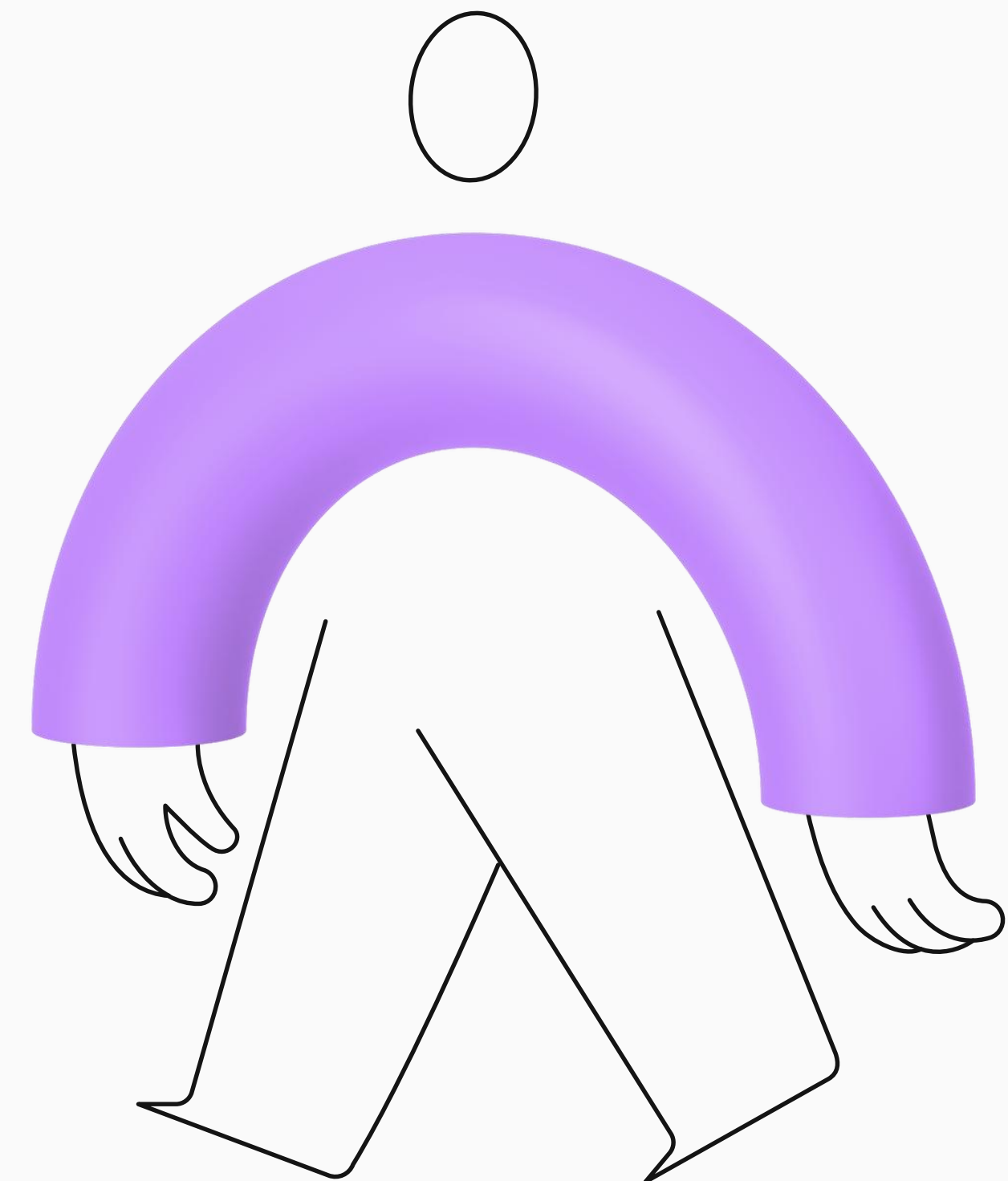
СУБД поддерживает двусвязный список кадров. При каждом чтении страница переносится в «голову» списка. На вытеснение всегда идёт страница из «хвоста».



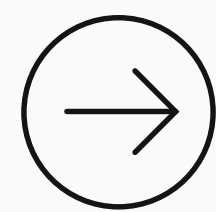
Уязвимость LRU

- **Сценарий:** выполнение тяжёлого аналитического запроса или резервного копирования, требующего полного сканирования таблицы (Full Table Scan), размер которой превышает объём Buffer Pool.
- **Эффект:** сканирование последовательно загружает новые страницы, мгновенно вымывая из оперативной памяти все исторически «горячие» индексы и часто используемые данные других транзакций.

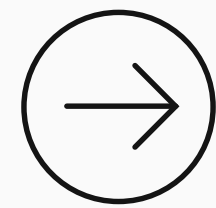
Итог: кеш полностью засоряется «мусором», который был прочитан ровно один раз и больше никогда не понадобится. Производительность всей системы падает до скорости дискового I/O.



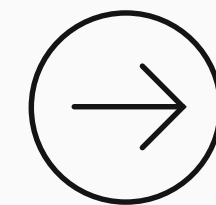
Алгоритм Clock



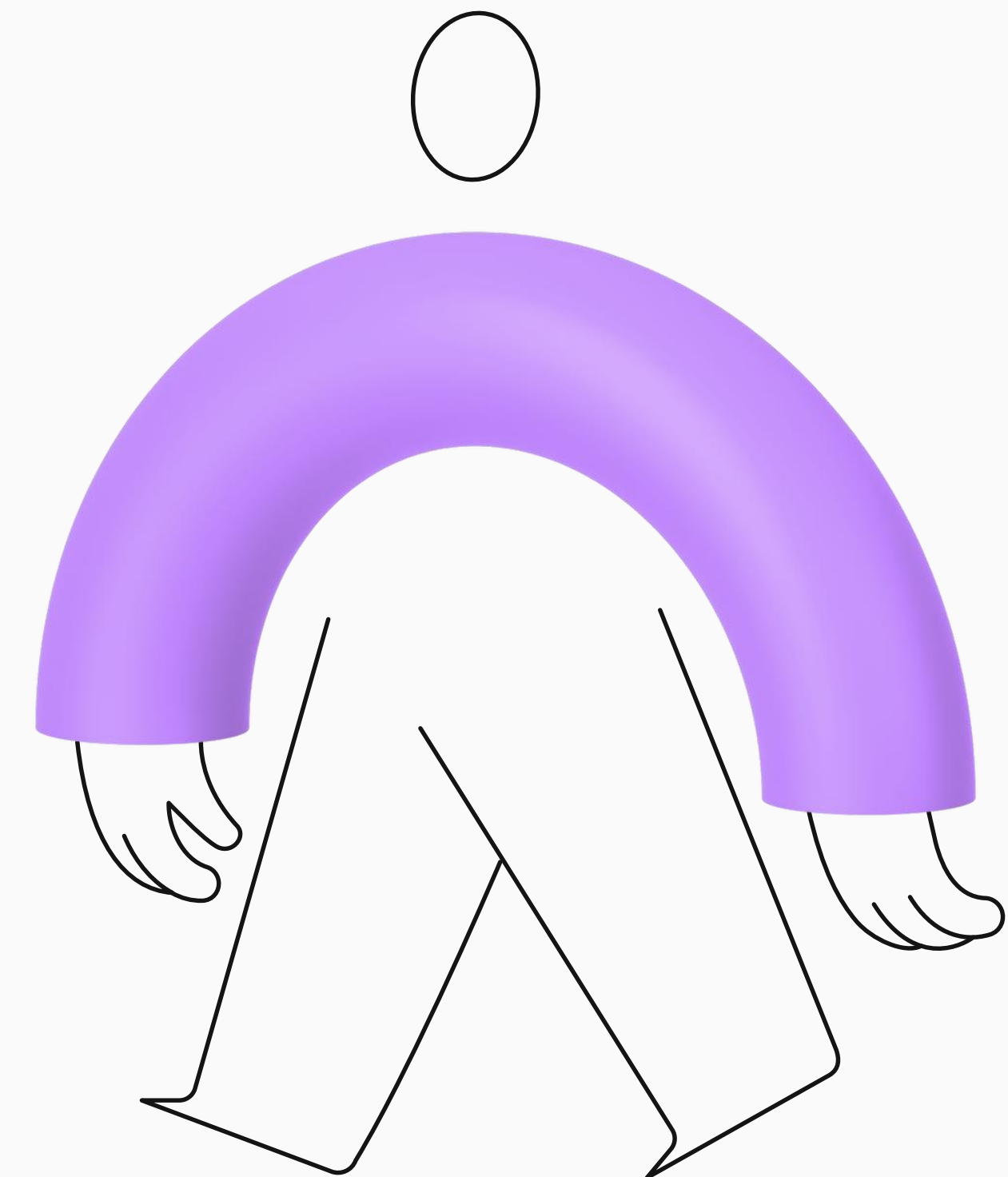
Механика: кадры организованы не в двусвязный список, а в кольцевой буфер. Каждому кадру присваивается Reference Bit (бит обращения).



Правило доступа: при любом обращении к странице её бит ставится в 1.



Стрелка часов (Sweep / Clock Hand): фоновый процесс циклично обходит кадры. Если видит 1, сбрасывает в 0 (даёт странице «второй шанс») и идёт дальше. Если видит 0, вытесняет страницу.

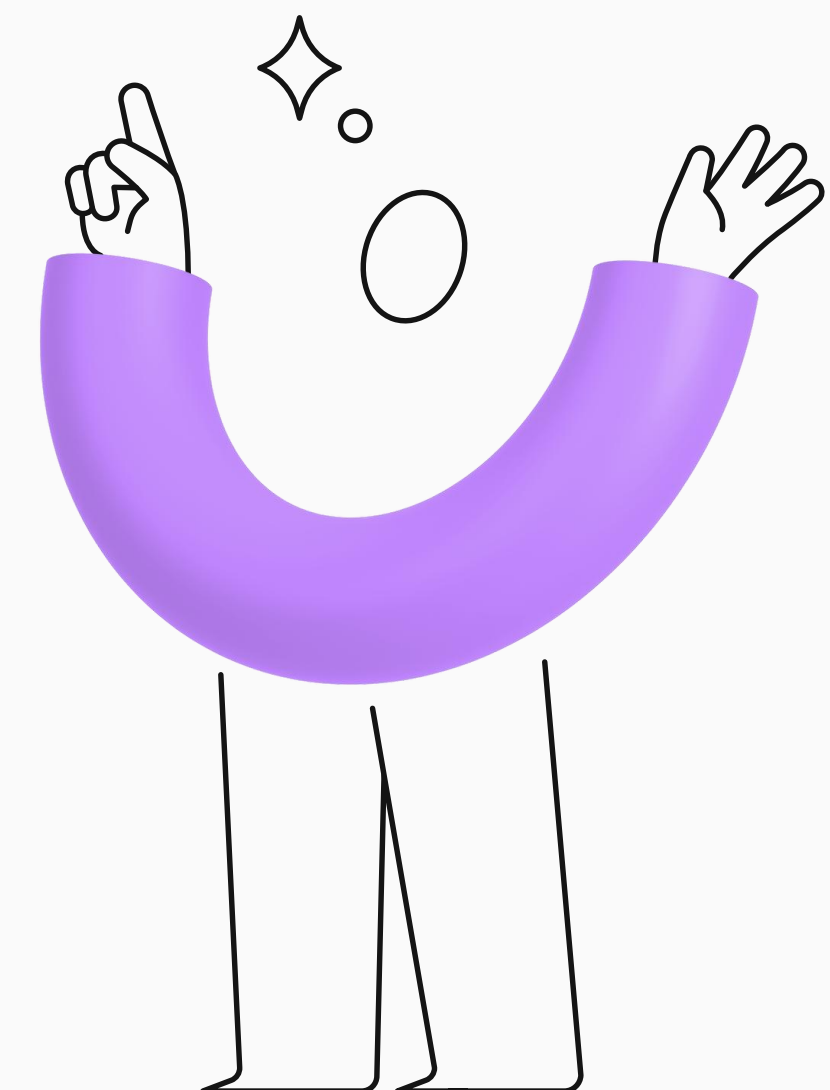


Защита от сканирований

Развитие алгоритма Clock для промышленных СУБД

- **Суть решения:** страница не признаётся «горячей» при первом же чтении.
- **Разделение очередей:** буфер логически делится на зону «карантина» (для новых страниц) и зону «горячих» данных.
- **Правило:** чтобы попасть в защищённую горячую зону и не быть вытесненной, к странице нужно обратиться как минимум дважды (концепция LRU-K).

Sequential Flooding — транзитный мусор от полного сканирования таблиц умирает в зоне карантина, не вытесняя полезные индексы.



Матрица политик буфера

	No Steal	Steal
No Force		Fastest
Force	Slowest	

	No Steal	Steal
No Force	No UNDO REDO	UNDO REDO
Force	No UNDO No REDO	UNDO No REDO

Steal vs No-Steal

		No Steal	Steal			No Steal	Steal
No Force	Force		Fastest	No Force	Force	No UNDO REDO	UNDO REDO
		Slowest				No UNDO No REDO	UNDO No REDO

Force vs No-Force

		No Steal	Steal
No Force			Fastest
Force	Slowest		

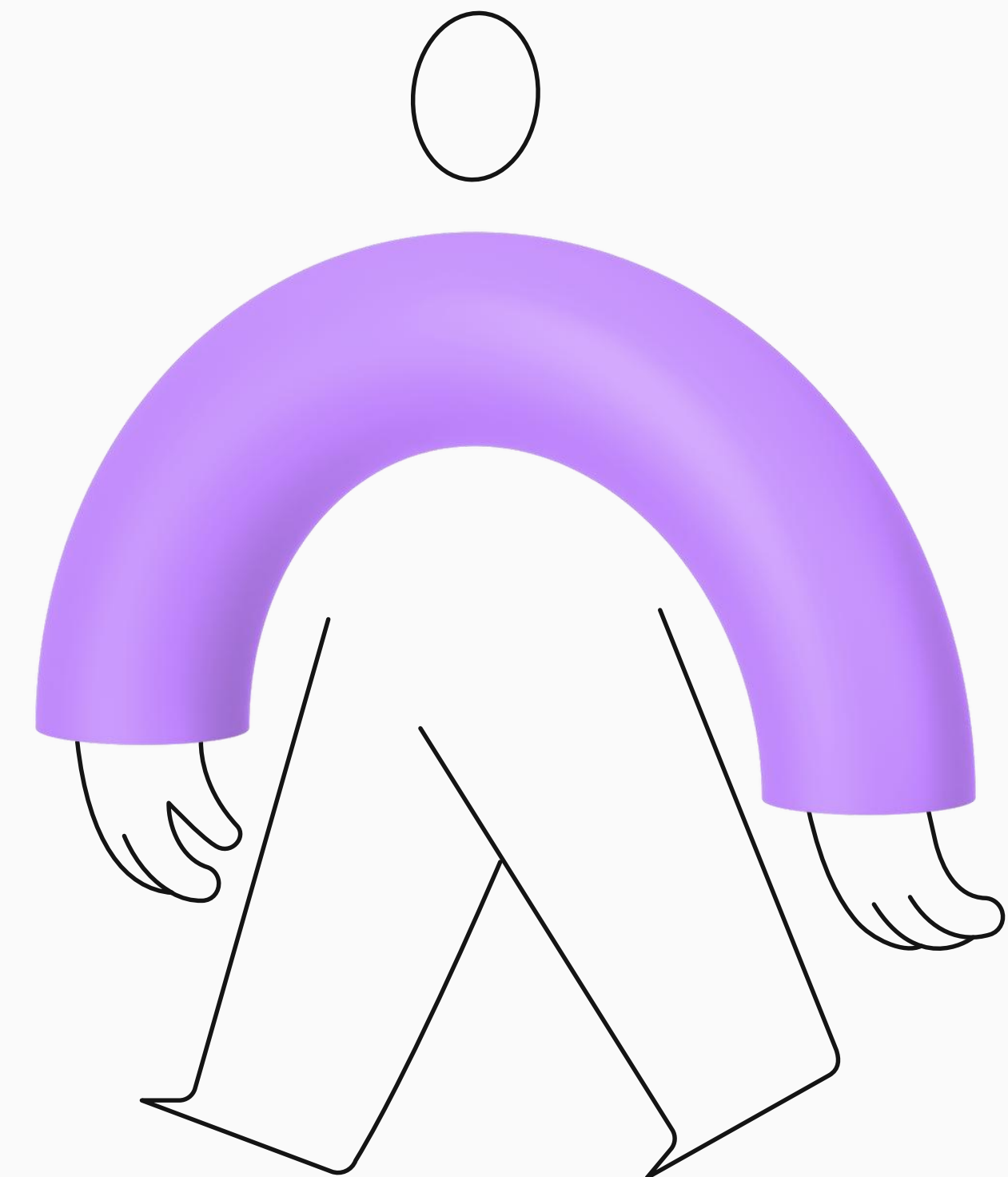
		No Steal	Steal
No Force	No UNDO REDO	UNDO REDO	
Force	No UNDO No REDO	UNDO No REDO	

Steal/No-Force

Абсолютно все современные реляционные движки (PostgreSQL, Oracle, SQL Server, InnoDB) используют политику Steal/No-Force.

Это единственная комбинация, дающая максимальную I/O-производительность и поддержку транзакций неограниченного размера.

Сама по себе связка Steal/No-Force нарушает гарантии Atomicity и Durability. Для устранения этой уязвимости СУБД обязана использовать внешний независимый лог — журнал предзаписи (WAL), который обеспечит недостающие функции UNDO и REDO.

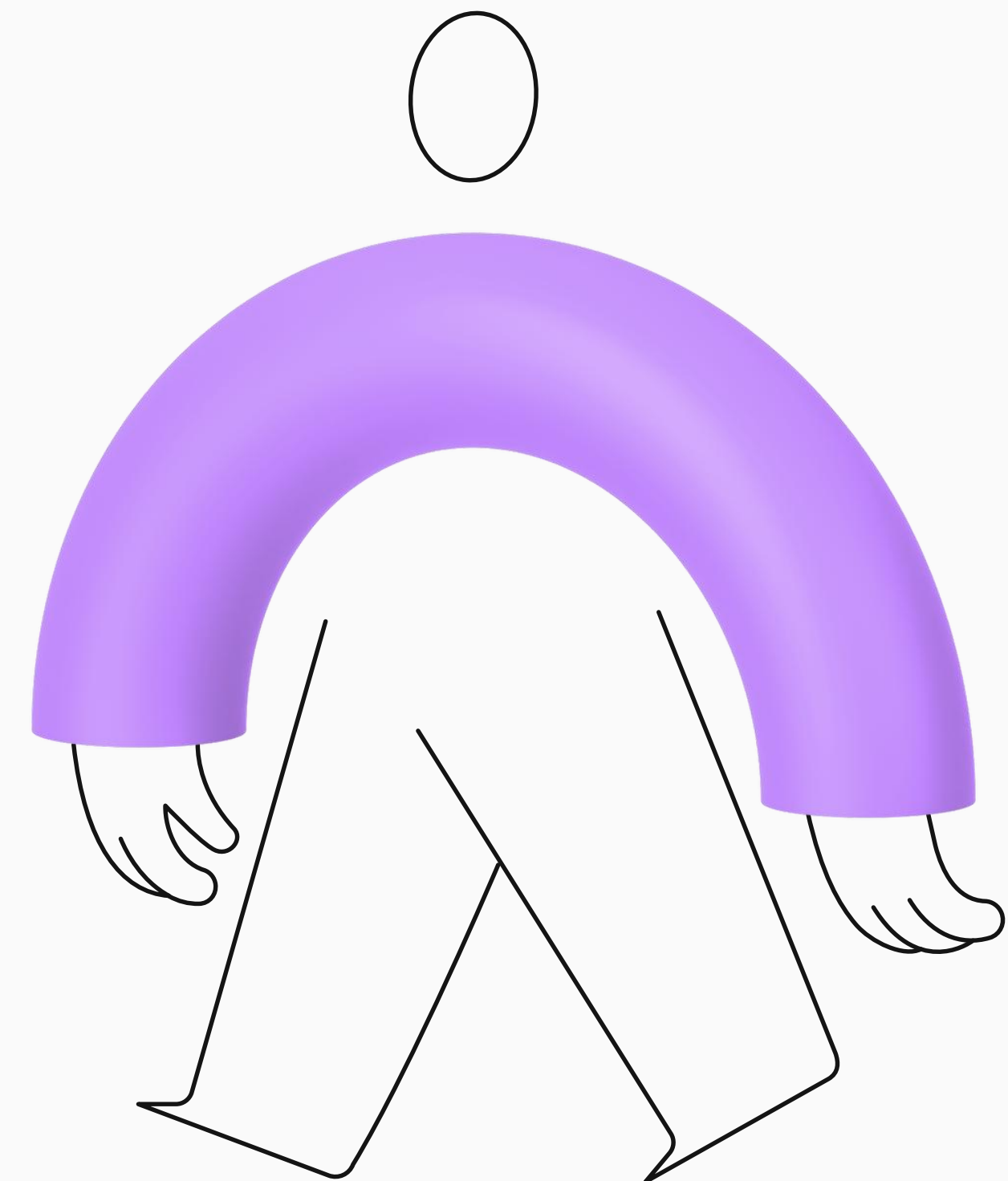


Write-Ahead Logging (WAL)

Мы используем политику Steal/No-Force (данные могут быть не записаны при коммите, а мусор от чужих транзакций может попасть на диск).

Журнал предзаписи (WAL) — это отдельный файл, в который строго последовательно (Append-Only) записываются все изменения базы данных.

Запись об изменении в журнале обязана быть физически сброшена (fsync) на энергонезависимый накопитель до того, как изменённая страница данных будет вытеснена из оперативной памяти на диск.

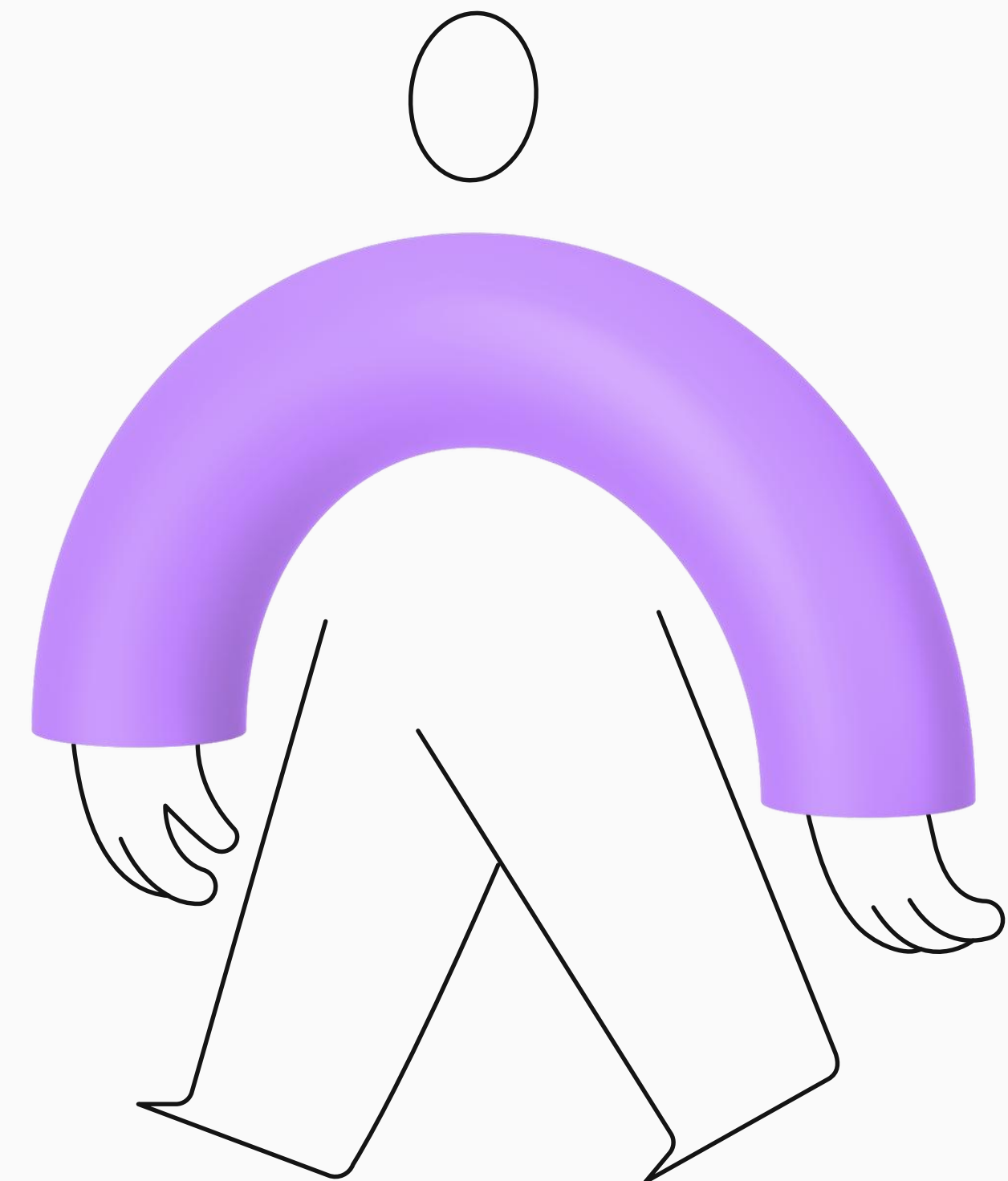


Log Sequence Number (LSN)

LSN — это глобальный, монотонно возрастающий 64-битный идентификатор. Каждая новая запись в журнале WAL получает свой уникальный LSN.

LSN можно воспринимать как логическое время внутри базы данных или как физическое смещение (offset) записи в байтах от начала лог-файла.

Позволяет СУБД безошибочно определять, какие операции уже применены к конкретной странице данных, а какие — ещё нет.

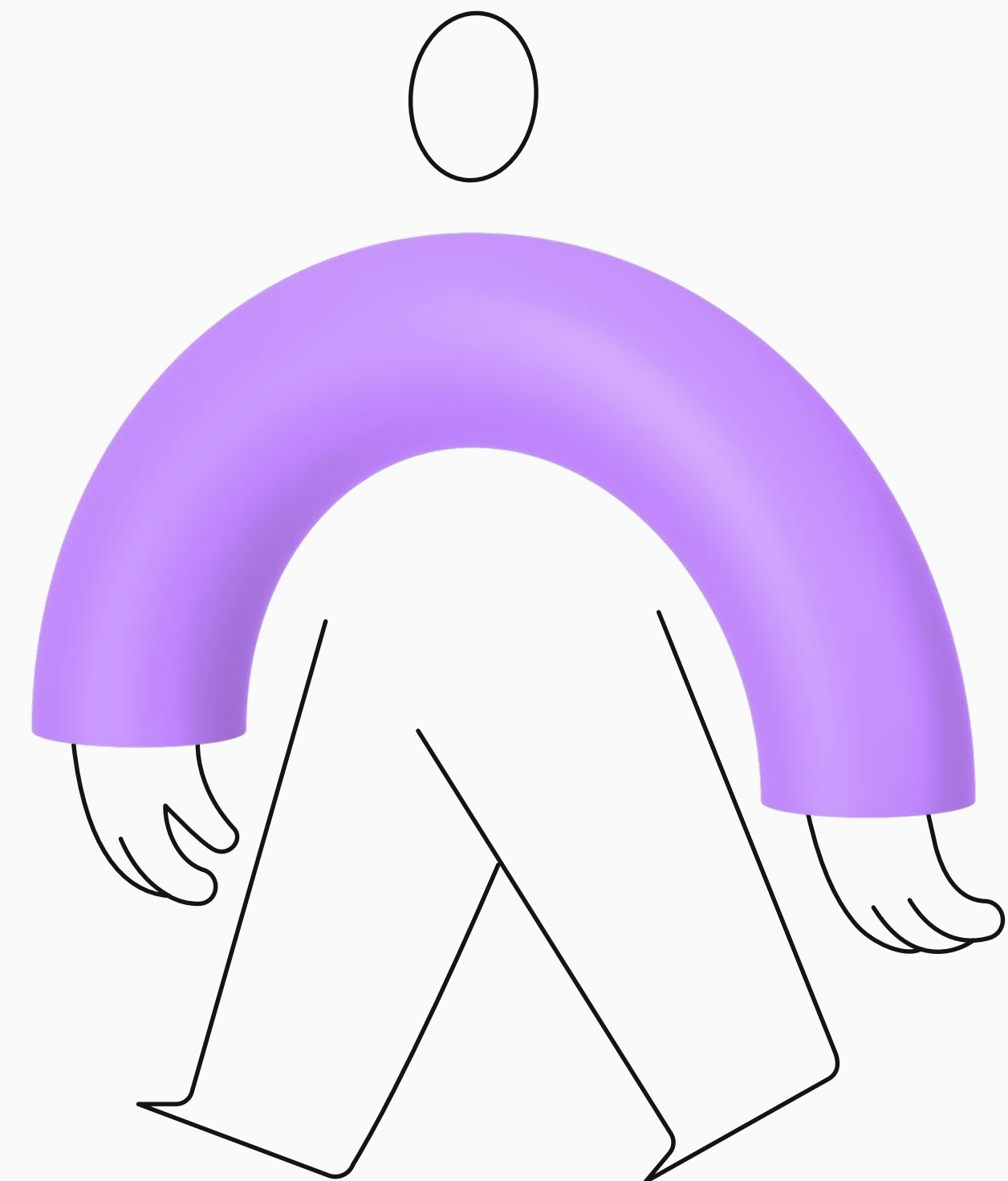


PageLSN и FlushedLSN

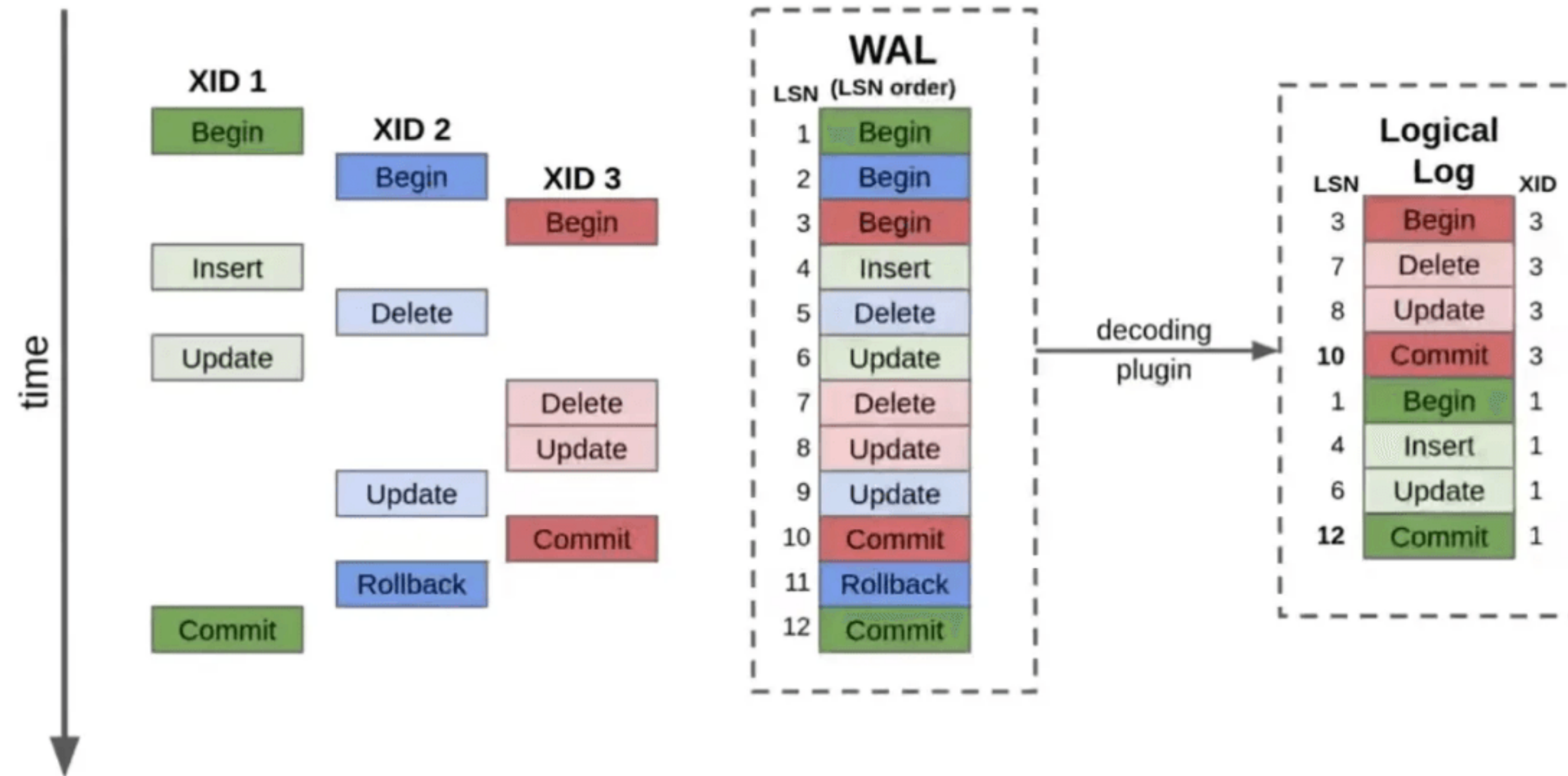
PageLSN: хранится в заголовке каждой страницы данных. Обновляется в оперативной памяти каждый раз, когда транзакция изменяет эту страницу (записывается LSN операции, которая сделала изменение).

FlushedLSN: глобальная переменная в памяти СУБД. Хранит максимальный LSN, который на данный момент гарантированно и физически сохранён на диске в файле WAL.

Буферный менеджер имеет право вытеснить грязную страницу на диск только если выполняется условие: $\text{PageLSN} \leq \text{FlushedLSN}$. Если условие не выполнено, СУБД принудительно инициирует сброс хвоста WAL на диск.



Логирование



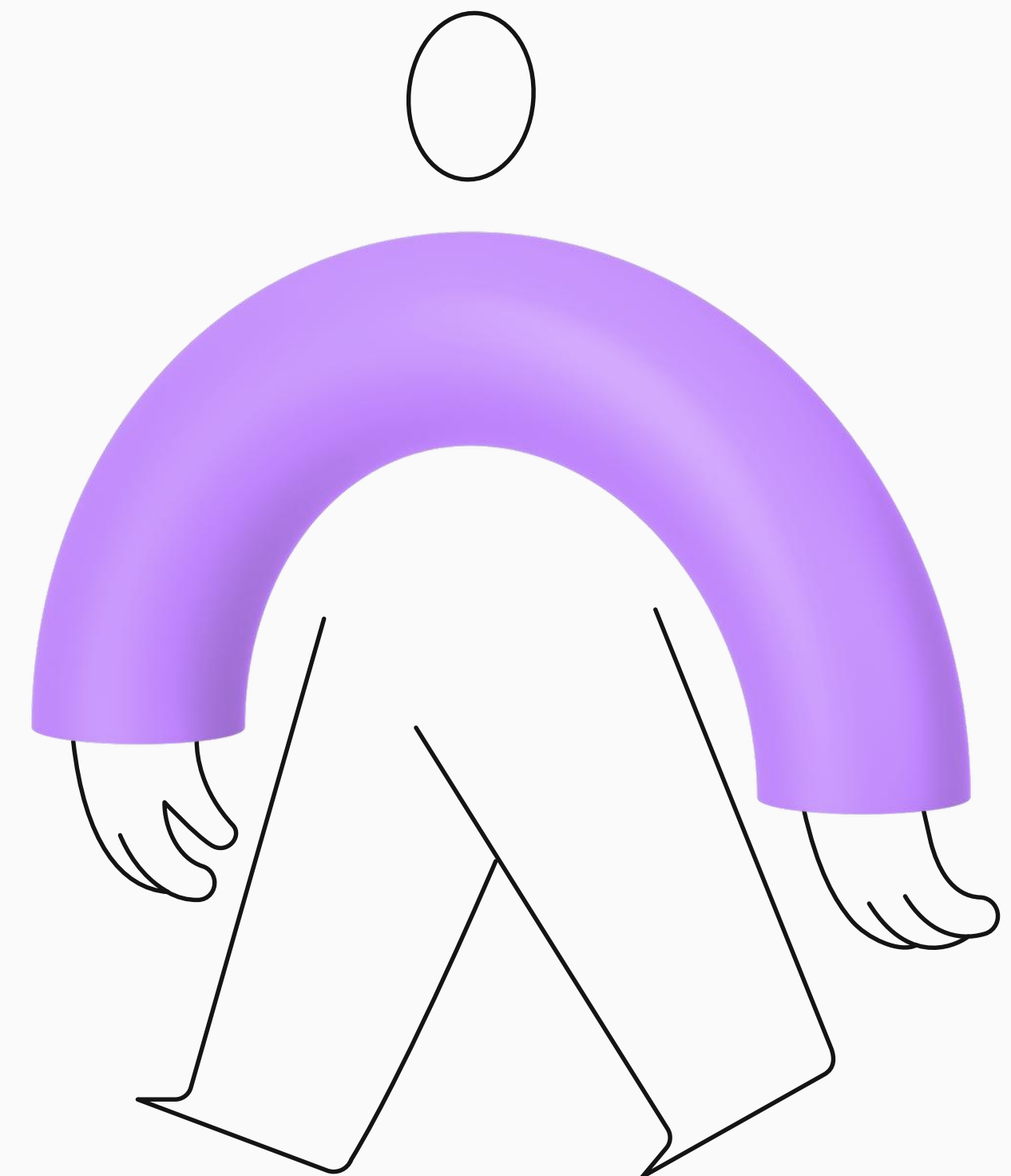
Разорванные страницы

СУБД оперирует страницами данных по 8 Кб (PostgreSQL) или 16 Кб (InnoDB). Диски и операционная система гарантируют атомарность записи только на уровне сектора в 512 байт (или блока в 4 Кб для современных SSD).

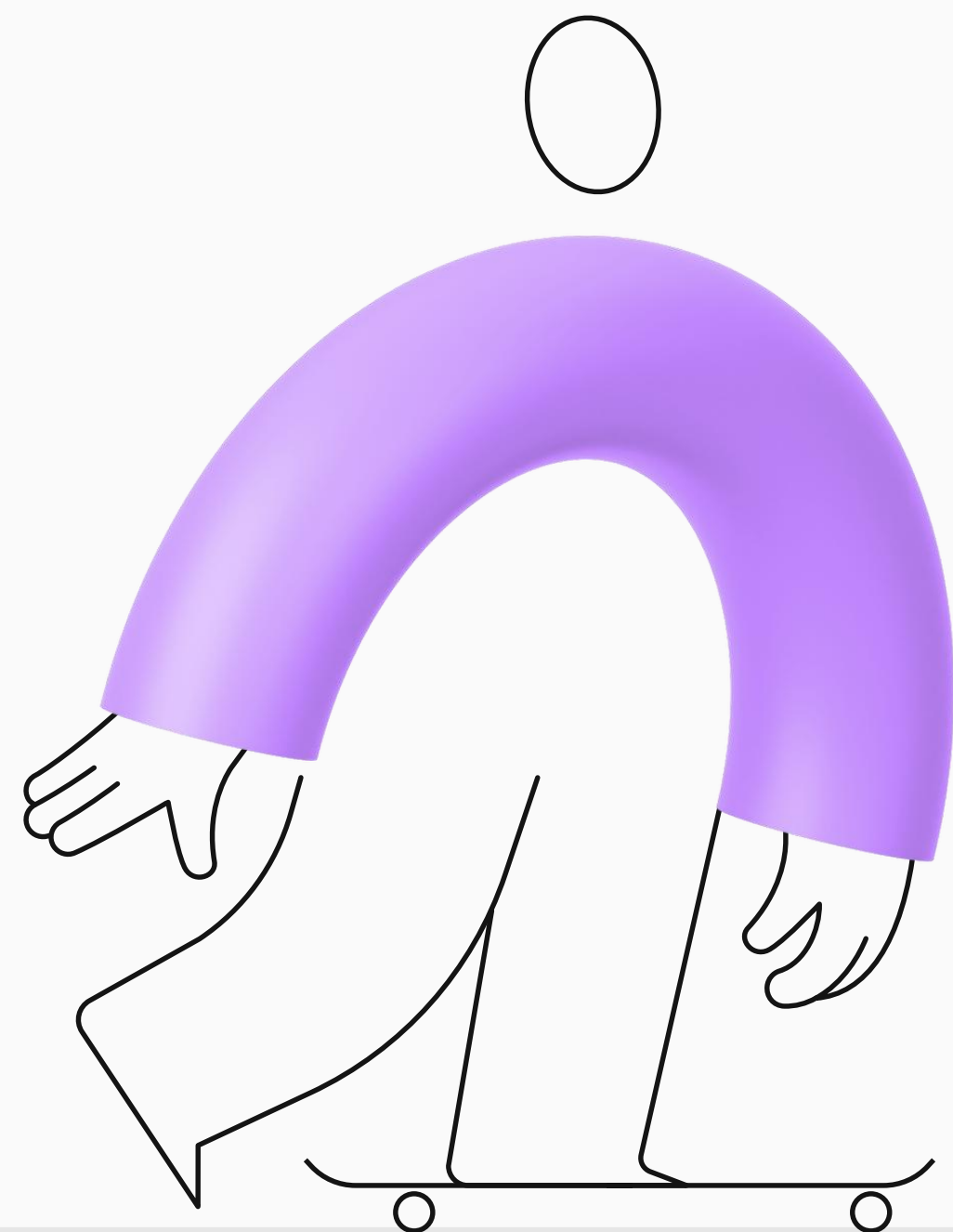
СУБД начинает сброс грязной 8 Кб страницы на диск. Сервер обесточивается на середине процесса.

Итог — на диске оказывается «химера»: первые 4 Кб содержат новые данные, а вторые 4 Кб — старые. Контрольная сумма (checksum) страницы не совпадает.

Данные безвозвратно повреждены, так как WAL не сможет корректно накатить изменения на битую базовую страницу.



Защита от разорванных страниц



Full Page Writes (PostgreSQL)

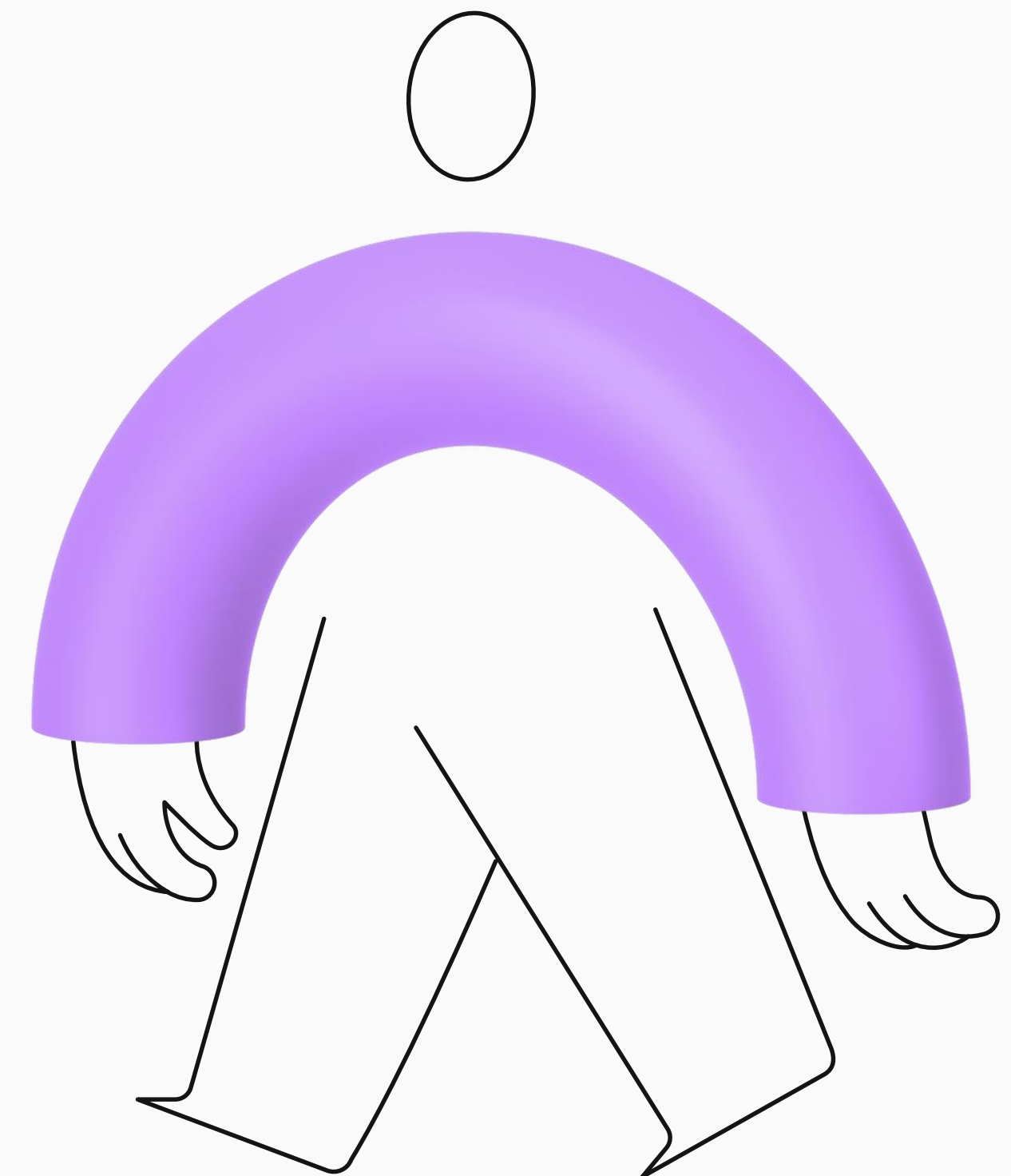
При первом изменении страницы после контрольной точки в WAL записывается не логическая дельта, а полная физическая копия всей страницы (8 Кб). При сбое СУБД просто заменяет битую страницу на диске её эталонной копией из журнала.

Doublewrite Buffer (InnoDB/MySQL)

Выделенная последовательная зона на диске. Сначала СУБД пишет грязные страницы туда (строго последовательно), делает fsync и только потом разносит эти же страницы по реальным файлам данных. Если сервер падает на втором шаге, неповреждённая страница извлекается из буфера двойной записи.

Crash Recovery

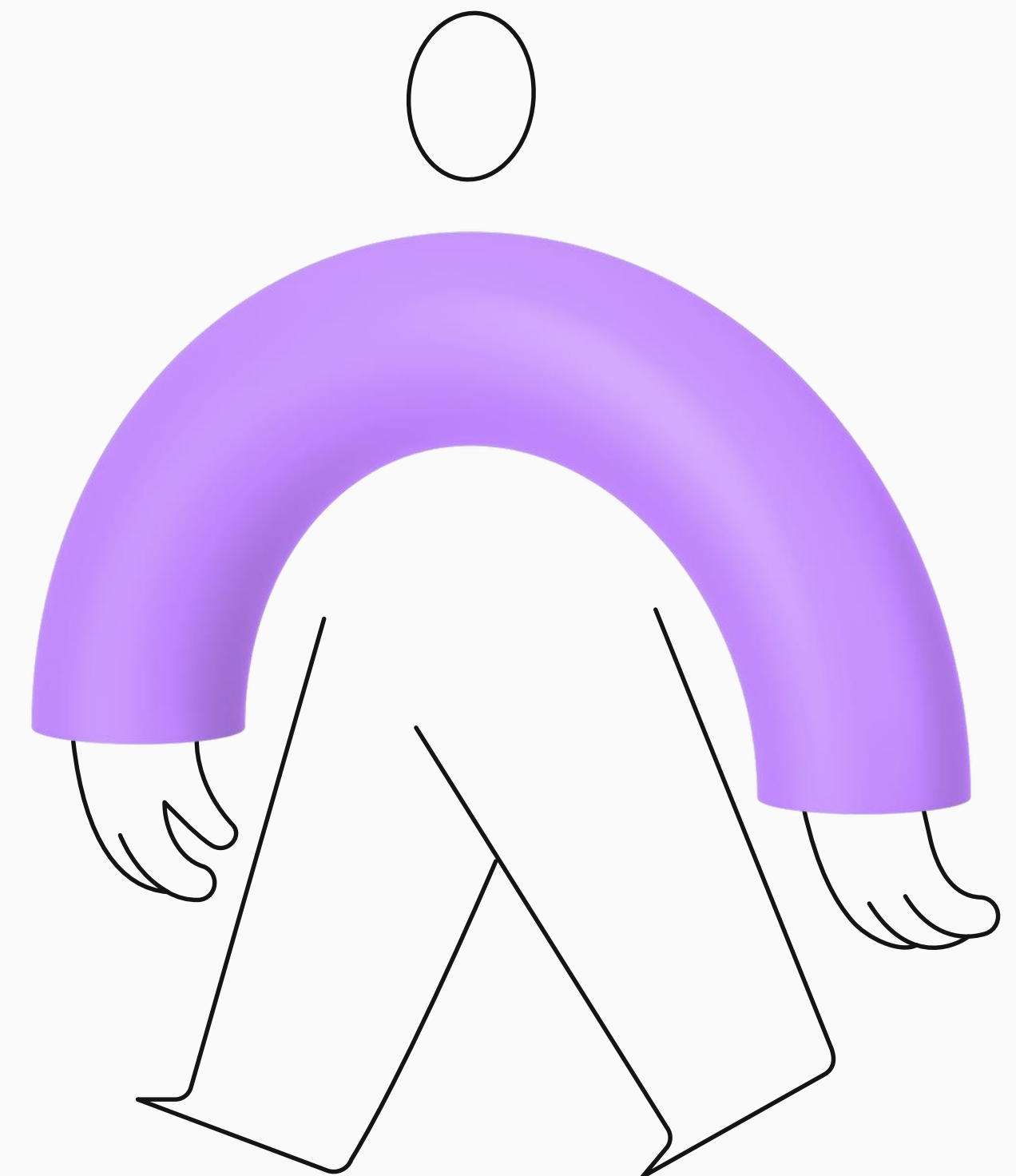
- В штатном режиме работы СУБД журнал WAL работает по принципу «только запись». Никто не читает WAL при выполнении обычных SELECT или UPDATE.
- WAL читается только один раз — при перезапуске сервера после падения.
- Диск после сбоя находится в физически неконсистентном состоянии (там есть мусор от незавершённых транзакций и не хватает данных от подтверждённых).
- Задача восстановления — опираясь на WAL, точно воссоздать состояние оперативной памяти на микросекунду до сбоя, а затем откатить лишнее.



Checkpoints

Журнал растёт бесконечно, а время восстановления (RTO — Recovery Time Objective) должно быть минимальным. Поэтому есть механизм Checkpoint — это фоновый процесс, который принудительно сбрасывает все текущие «грязные» страницы из оперативной памяти на диск и делает пометку в WAL.

Контрольная точка устанавливает границу — Redo LSN. Это означает: «Все изменения до этой точки 100% находятся на диске». Старые сегменты WAL больше не нужны для восстановления после сбоя и могут быть удалены или перезаписаны.



Протокол ARIES

Введение в классический алгоритм восстановления, разработанный в IBM Research, который лежит в основе большинства современных баз данных.

ARIES — Algorithm for Recovery and Isolation Exploiting Semantics.

Основной принцип — «Repeating History» (повторение истории). Сначала мы безусловно воссоздаём картину аварии и только потом наводим порядок.

Три фазы работы

1. Фаза анализа (Analysis)
2. Фаза повтора (Redo)
3. Фаза отмены (Undo)

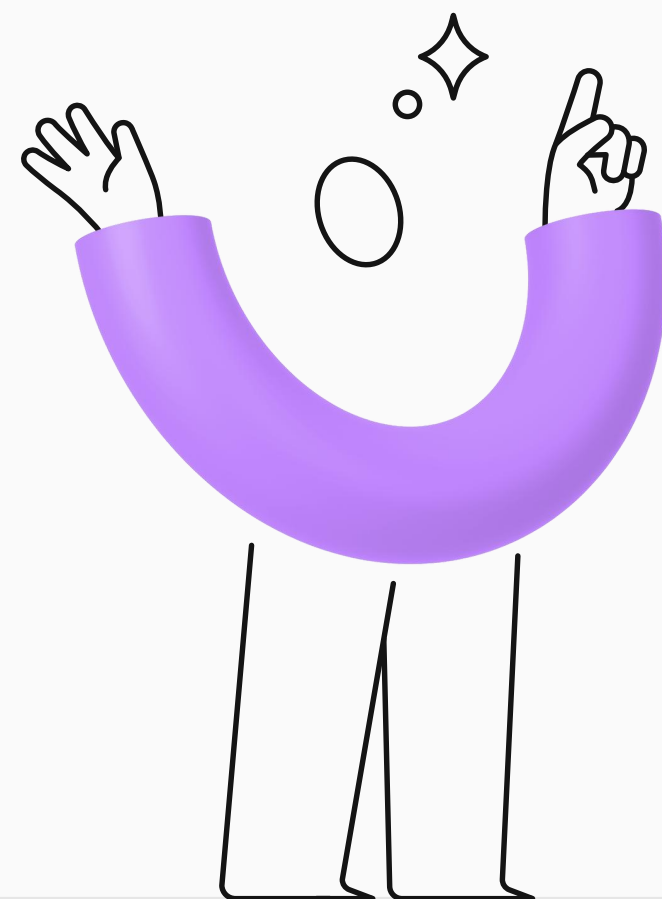


ARIES

Фаза анализа (Analysis Phase)

Первый шаг алгоритма при старте упавшей СУБД.

СУБД читает WAL вперёд (от последнего успешного Checkpoint до конца лога).



Результат 1: воссоздаётся

«Таблица транзакций» (Transaction Table).

Мы узнаём, какие транзакции были активны в момент сбоя и не успели сделать COMMIT (транзакции-неудачники, losers).

Результат 2: воссоздаётся «Таблица грязных страниц» (Dirty Page Table). Мы определяем точный LSN, с которого нужно начинать чтение лога для наката данных (Redo LSN), чтобы не делать лишнюю работу.

ARIES

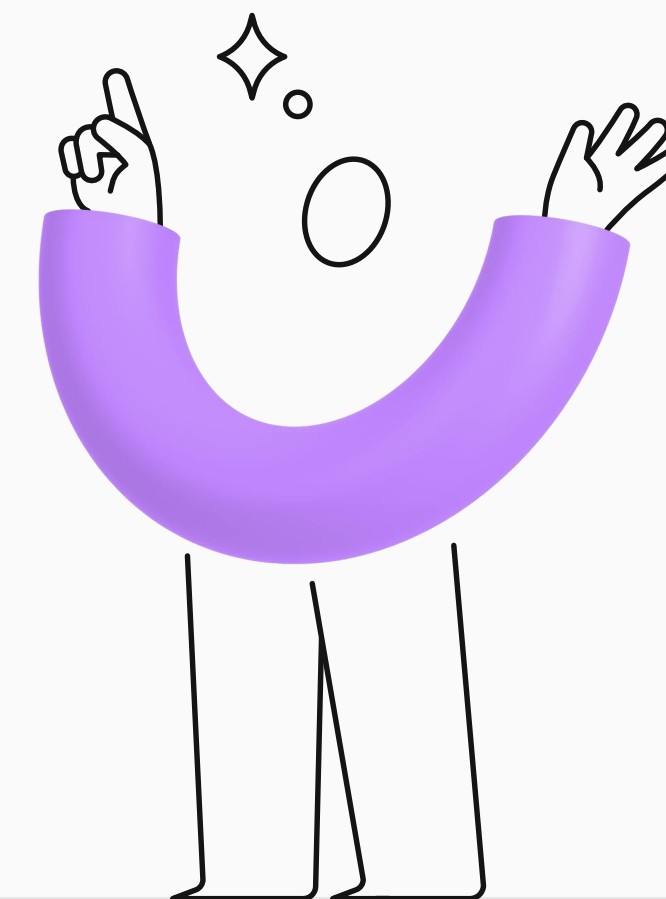
Фаза повтора (Redo Phase)

Второй шаг алгоритма — реализация принципа «Repeating History».

СУБД читает WAL вперёд, начиная с Redo LSN, и безусловно применяет к страницам данных все зафиксированные изменения.

Изменение из лога применяется к странице данных, только если $\text{LogRecordLSN} > \text{PageLSN}$.

На миллисекунду база данных возвращается в точное состояние, в котором она находилась за мгновение до отключения питания, включая весь мусор от незавершённых транзакций.



ARIES

Фаза отмены (Undo Phase)

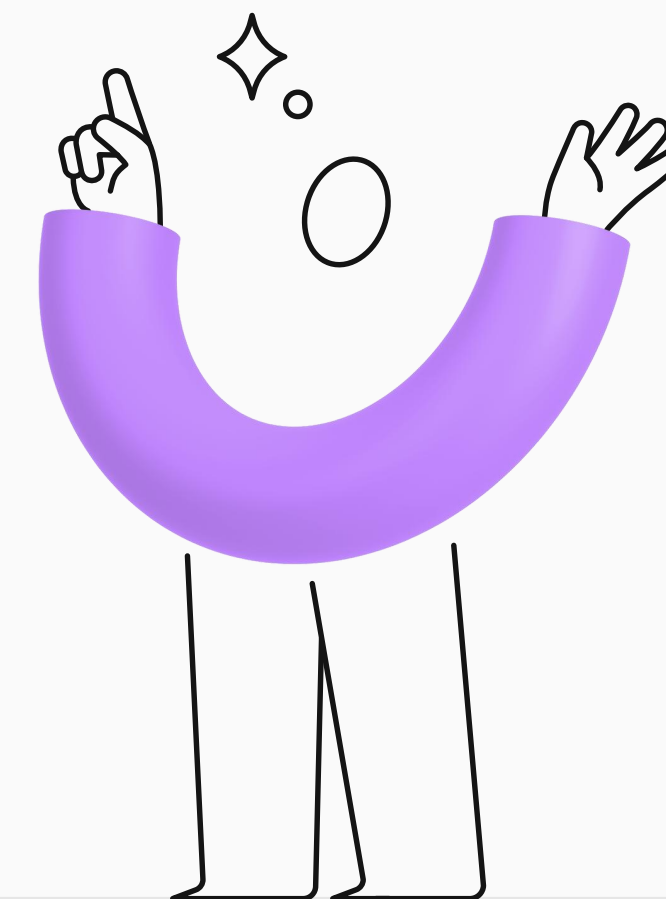
Третий, финальный шаг — обеспечение гарантии атомарности.

СУБД читает WAL в обратном порядке, начиная с конца лога.

Для всех транзакций-неудачников, найденных на фазе анализа, выполняются обратные операции (логическое вычитание).

CLR (Compensation Log Records): в ARIES сами операции отмены тоже записываются в WAL.

Если во время фазы отмены сервер снова упадёт, при следующем старте СУБД не будет откатывать транзакцию дважды, а продолжит с того места, где остановилась.



Резюме

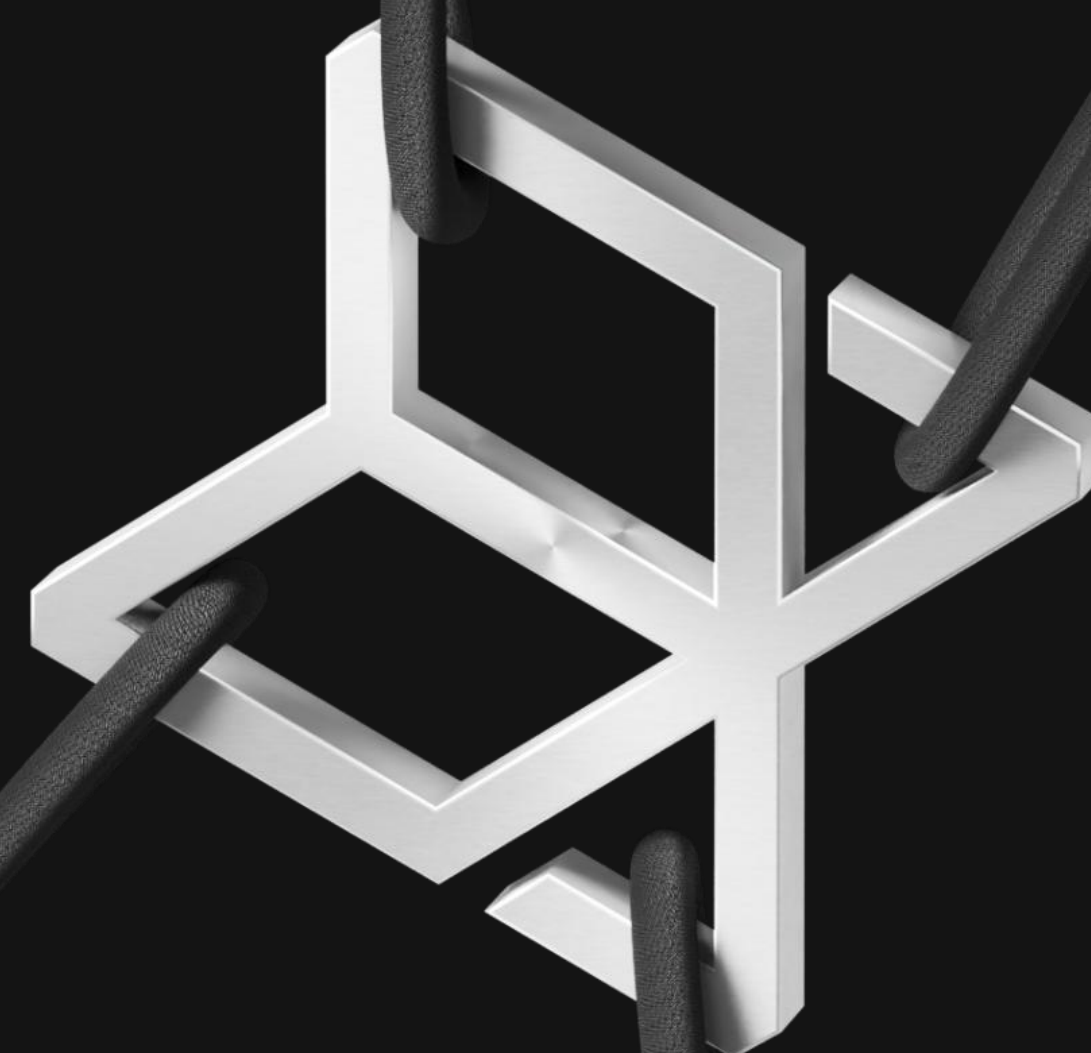


→ Buffer Pool Manager даёт нам производительность.

→ Протокол WAL даёт нам надёжность.

→ Log Sequence Number (LSN) — математический механизм, который жёстко синхронизирует летучую память и энергонезависимый диск.

Вопросы



Минутка обратной связи

